Checkpoint1: Motion & Odometry

Peilin Meng, Yijie Liao, Yun Jong Na, Yunzhu Deng
University of Michigan

I. Characterize Wheel Speed for an Open Loop Controller

A. Motor Calibration Setup

Motor calibration was conducted on three different surfaces in the following order:

- classroom carpet
- tile floor
- foam mat

For each surface, the motor calibration routine estimated a linear mapping from wheel speed to PWM command:

$$\text{PWM} = m \cdot v + b \tag{1}$$

Separate parameters were obtained for left and right wheels.

B. Calibration Parameters

TABLE I: Parameters on classroom carpet.

Wheel	**Direction**	**Slope** m	**Intercept** b
Left	Positive	0.070916	0.072829
Left	Negative	0.065925	-0.099292
Right	Positive	0.067609	0.090518
Right	Negative	0.073808	-0.092499

TABLE II: Parameters on tile floor.

Wheel	**Direction**	**Slope** m	**Intercept** b
Left	Positive	0.070978	0.060307
Left	Negative	0.063589	-0.087558
Right	Positive	0.065160	0.084599
Right	Negative	0.076408	-0.071913

TABLE III: Parameters on foam mat.

Wheel	**Direction**	**Slope** m	**Intercept** b
Left	Positive	0.057799	0.058034
Left	Negative	0.051501	-0.094726
Right	Positive	0.052931	0.074496
Right	Negative	0.059116	-0.085079

C. Motor Speed vs. PWM Plot

For each wheel, we plot PWM versus motor speed under loaded-wheel conditions and overlay the fitted linear model, as shown in Figures 1 and 2.

D. Square Path Odometry Experiment (Example Data)

The robot is commanded to execute the following motion sequence:

1) Drive forward for 1 meter
2) Rotate counter clockwise by 90° (approximately 1.57 rad)
3) Drive forward for 1 meter

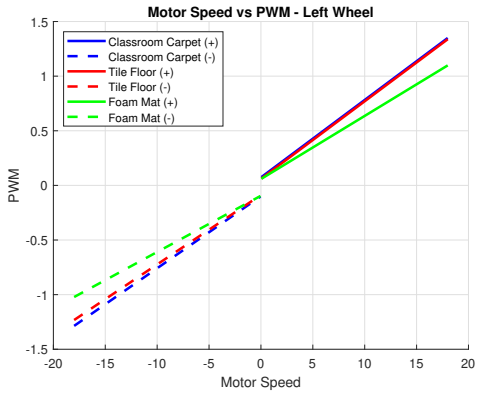

Fig. 1: Motor speed versus PWM for the left wheel on three different surfaces. Solid lines indicate positive direction, while dashed lines indicate negative direction.

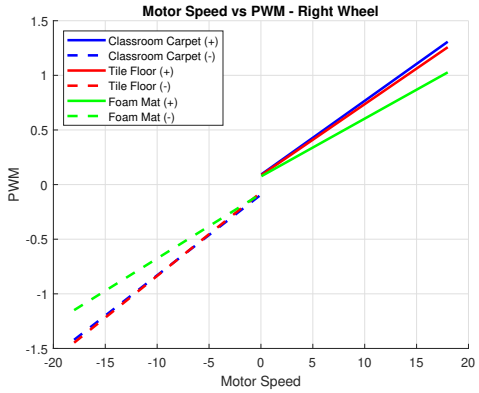

Fig. 2: Motor speed versus PWM for the right wheel on three different surfaces. Solid lines indicate positive direction, while dashed lines indicate negative direction.

1) Measured Final Odometry: Measured final pose:

$$(x, y, \theta)_{\text{meas}} = (0.84 \text{ m},\ 1.38 \text{ m},\ 1.42 \text{ rad})$$

Ideal final pose:

$$(x, y, \theta)_{\text{ideal}} = (1.00 \text{ m},\ 1.00 \text{ m},\ 1.57 \text{ rad})$$

2) Difference from Ideal:

$$\Delta x = x_{\text{meas}} - 1.00 = -0.16 \text{ m}$$

$$\Delta y = y_{\text{meas}} - 1.00 = +0.38 \text{ m}$$

$$\Delta \theta = \theta_{\text{meas}} - 1.57 = -0.15 \text{ rad}$$

E. Discussion

1) Left vs. Right Wheel Variation: Across all surfaces, the left and right wheels show consistent differences in both slope and intercept. For instance, on the tile floor, the negative-direction slope of the right wheel ($m_{\text{rn}} = 0.076408$) is noticeably larger than that of the left wheel ($m_{\text{ln}} = 0.063589$). This asymmetry can introduce heading drift during straight motion and bias during turns.

2) Variation Across Surfaces: The foam mat exhibits smaller slope values overall compared to the carpet and tile floor. This indicates higher rolling resistance and stronger low-speed friction effects on the foam surface. In contrast, the harder surfaces allow more efficient wheel motion for the same PWM input.

3) Possible Sources of Variation: The observed variations are mainly due to surface friction differences and mechanical asymmetry between the two wheels. Additional factors include battery voltage variation during calibration and the limitations of a linear motor model at low speeds.

4) Reasons for Odometry Error: Odometry errors primarily arise from small heading errors during the 90° rotation, which propagate into position errors. Wheel slip during turning and unequal wheel responses further contribute to the final pose error.

II. Wheel Speed Control Using PID Controllers

While open-loop motor calibration provides a nominal mapping from desired wheel speed to PWM command, it cannot compensate for disturbances such as surface friction changes, battery voltage drop, or mechanical asymmetry. To improve velocity tracking performance, we evaluated several closed-loop PID-based control modes provided in `mbot_classic_ros.c` and tuned the corresponding controller parameters.

A. Controller Modes Overview

The firmware provides multiple controller modes for wheel and body velocity control. In this section, we focus on the following modes implemented in the `switch (local_cmd.drive_mode)` loop:

- `MODE_MOTOR_PWM`: direct open-loop PWM control
- `MODE_MOTOR_VEL`: wheel velocity control using feedforward, PID, or combined FF+PID
- `MODE_MBOT_VEL`: body velocity control converted to wheel velocities using differential drive kinematics

B. Wheel Velocity PID Control Formulation

For each wheel, the PID controller regulates the error between the commanded wheel angular velocity and the measured wheel velocity.

The wheel velocity tracking error is defined as:

$$e(t) = \omega_{\text{cmd}}(t) - \omega_{\text{meas}}(t) \tag{2}$$

The PID correction term is computed as:

$$u_{\text{PID}}(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt} \tag{3}$$

Depending on the selected control mode, the final PWM command is given by:

$$u = \begin{cases} u_{\text{FF}}, & \text{FF only} \\ u_{\text{PID}}, & \text{PID only} \\ u_{\text{FF}} + u_{\text{PID}}, & \text{FF + PID} \end{cases} \tag{4}$$

where u_{FF} is obtained from the calibrated open-loop mapping described in Section 1.1.

C. PID Parameter Tuning Procedure

PID parameters were tuned experimentally using a pragmatic and stability-driven approach. Rather than aggressively tuning all three gains, priority was placed on achieving stable and repeatable wheel velocity tracking under real operating conditions.

In practice, the tuning procedure proceeded as follows:

1) The proportional gain K_p was increased gradually until the wheel velocity exhibited a fast and responsive rise to step commands, while avoiding sustained oscillations or audible motor chatter.
2) Although steady-state error was occasionally observed, increasing the integral gain K_i often led to instability, overshoot, or oscillatory behavior due to encoder noise and actuator saturation. As a result, K_i was kept very small or set to zero in most experiments.
3) The derivative gain K_d was either kept small or omitted, as excessive derivative action amplified discretization noise in the wheel velocity estimates and did not provide consistent performance improvements.

Instead of relying on integral action to correct long-term errors, asymmetries between the left and right wheels were addressed by independently adjusting the proportional gains for each wheel. This approach proved effective in improving straight-line motion and achieving more accurate in-place rotations, such as 90° turns.

The primary tuning metrics included rise time, oscillatory behavior, repeatability across runs, and the accuracy of rotational maneuvers, rather than minimizing steady-state error alone.

D. PID Parameters Tested

PID tuning was conducted experimentally using a step response test. A custom testing script was implemented to command a constant forward velocity of 0.5 m/s for 5 seconds. For each test, wheel velocities, controller outputs, and odometry were recorded using ROS bag files and analyzed offline. Figure 3 shows a representative tuning result for the feedforward-only (FF) controller. The PID-only and FF+PID controllers were tuned and evaluated using the same experimental procedure, with corresponding results discussed in the following sections.

The tested gain configurations are summarized in Table IV, while qualitative observations are discussed below using representative step response results.

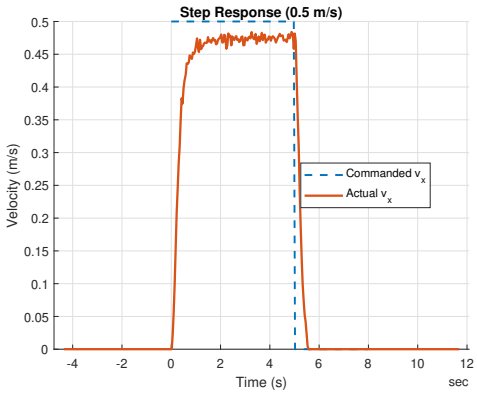

Fig. 3: Feedforward-only control. Commanded versus measured wheel velocity during a 0.5 m/s step input after tuning.

TABLE IV: Controller modes and PID gain values evaluated during tuning.

Test ID	Mode	K_p	K_i	K_d
1	FF	0.10	0.00	0.001
2	FF	0.15	0.00	0.001
3	PID	0.10	0.00	0.001
4	PID	0.15	0.00	0.001
5	FF+PID	0.20	0.00	0.001
6	FF+PID	0.22 (L) / 0.20 (R)	0.00	0.001
7	FF+PID	0.25 (L) / 0.22 (R)	0.00	0.001

1) Feedforward Only (FF): Using the default gain configuration ($K_p = 0.10$), the feedforward-only controller produced stable motor commands but exhibited a slow velocity response. During the 0.5 m/s step input, the measured wheel velocities remained consistently below the commanded value, resulting in under-travel within the 5 second test window.

Increasing the proportional gain to $K_p = 0.15$ improved the initial rise time slightly. However, the steady-state velocity error persisted, and the robot still failed to reach the expected travel distance. At higher gains, minor asymmetries between the left and right wheels began to appear, negatively affecting straight-line motion.

Overall, the FF-only controller was unable to compensate for unmodeled friction and load variations.

2) PID Only: Several PID-only configurations were tested to evaluate the effect of feedback control without feedforward compensation. Increasing K_p reduced rise time but made the controller increasingly sensitive to encoder noise. This sensitivity manifested as small oscillations in the wheel velocity estimates and corresponding PWM commands.

Attempts to introduce an integral term to eliminate steady-state error consistently resulted in unstable behavior, including overshoot and oscillations. Even with conservative gains, the PID-only controller exhibited poor repeatability across multiple runs.

As a result, PID-only control was deemed unreliable for consistent wheel speed regulation under real operating conditions.

3) Feedforward Plus PID (FF+PID): The combined FF+PID controller provided the best overall performance. With $K_p = 0.20$, wheel velocities closely tracked the commanded value with reduced steady-state error and improved rise time.

Although the initial FF+PID configuration improved distance tracking, slight asymmetry between the left and right wheels was still observed. This manifested as small heading deviations during straight-line motion.

To address this issue, the proportional gains for the left and right wheels were tuned independently. Increasing the left wheel proportional gain slightly improved balance, resulting in more consistent straight-line motion and more accurate distance tracking.

Further increasing K_p beyond this point did not yield noticeable performance gains and occasionally introduced mild oscillations. The final FF+PID configuration with asymmetric proportional gains was selected for subsequent experiments.

E. PID Error and Velocity Tracking Visualization

To quantitatively evaluate controller performance, we used the log data recorded by `test_wheel_pid.py`. The script applies a step command to the wheels and records both the commanded wheel velocities and the measured wheel velocities over time. Using these logs, we plotted commanded versus measured wheel velocity for the left and right wheels under three control modes: feedforward only (FF), PID only, and the combined controller (FF+PID). Figure 4 shows the left wheel comparison, and Figure 5 shows the right wheel comparison.

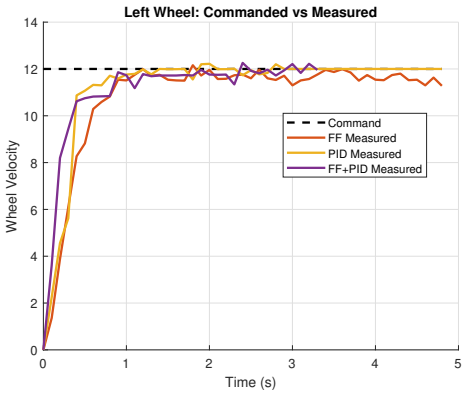

Fig. 4: Left wheel commanded versus measured wheel velocity for FF, PID, and FF+PID using `test_wheel_pid.py` logs.

Overall, the combined controller (FF+PID) achieved the best performance, exhibiting the fastest transient response and the smallest steady-state tracking error. The PID-only controller improved tracking compared to open-loop feedforward control but showed slower convergence and larger residual error. The feedforward-only controller had the weakest tracking performance, with the largest steady-state offset and reduced robustness to disturbances. In

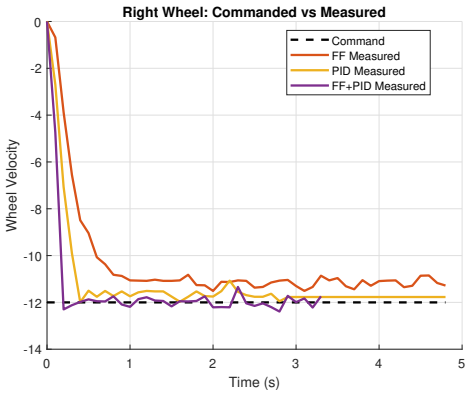

Fig. 5: Right wheel commanded versus measured wheel velocity for FF, PID, and FF+PID using `test_wheel_pid.py` logs.

summary, the observed performance ranking is **FF+PID** > **PID** > **FF** in terms of both response quality and stability.

F. Drive Square Experiment with PID Control

Using the tuned PID controller, the robot was commanded to execute a square trajectory four consecutive times. This experiment evaluates the ability of the controller to maintain straight-line motion and consistent turning behavior without explicit heading correction.

1) Experimental Setup: The motion sequence for each square consisted of:

1) Drive forward a fixed distance
2) Rotate in place by 90°
3) Repeat four times

A ROS bag file was recorded for this experiment for post-processing and analysis.

2) Qualitative Performance: Compared to open-loop control, the PID-based controller exhibited:

- Improved straight-line tracking
- Reduced wheel speed mismatch between left and right wheels
- More consistent turning behavior across repetitions

A video demonstrating the best-performing drive square experiment is provided as part of the submission.

G. Discussion

1) Comparison of Controller Modes: Open-loop feedforward control is simple and effective under nominal conditions but is sensitive to surface changes and disturbances. PID-only control improves robustness but may require larger integral gains to eliminate steady-state error. The combined FF+PID approach achieves the best performance by leveraging calibration for nominal behavior while using feedback to correct residual errors.

2) Tuning Criteria: The controller was considered sufficiently tuned when:

- Steady-state wheel velocity error was minimized
- Overshoot remained within acceptable bounds
- Oscillations were negligible
- The robot completed repeated square trajectories with minimal drift

These criteria reflect a balance between responsiveness and stability suitable for downstream navigation tasks.

III. Firmware Improvement

To further improve rotational accuracy and robustness during turning maneuvers, a firmware-level gyrodometry enhancement was implemented. Specifically, the wheel-based yaw rate estimate was replaced by a bias-corrected gyroscope measurement during odometry integration. This modification targets scenarios where wheel slip, asymmetry, or rapid angular accelerations degrade yaw rate estimation accuracy.

A. Experimental Design

Compared to the basic step response tests used for initial PID tuning, a more comprehensive command sequence was designed to stress both translational and rotational dynamics. A custom ROS node was implemented to publish a sequence of velocity commands to `/cmd_vel`, including forward and reverse steps, in-place rotations, and combined linear–angular motion.

The command sequence consists of three parts as shown in Figure 6.

- Forward and reverse velocity steps
- In-place rotations with large angular velocity commands
- Curved trajectories combining nonzero v_x and w_z

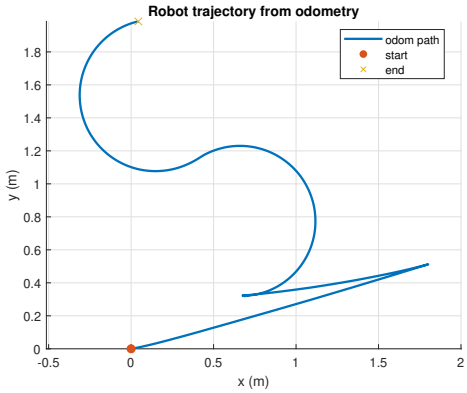

Fig. 6: Qualitative odometry trajectory during the step-based motion test. The path reflects a combination of straight-line segments, in-place rotations, and curved motion generated by the commanded v_x and w_z sequence.

B. Gyrodometry Implementation

The gyrodometry enhancement replaces the wheel-derived yaw rate with the IMU gyroscope measurement. To mitigate bias in the gyroscope signal, an online bias estimation procedure was applied during system startup. While the robot remains stationary, the yaw rate bias is estimated by averaging a fixed number of gyroscope samples. After bias convergence, all subsequent gyroscope measurements are corrected before being used for odometry integration.

C. Velocity Tracking Comparison

Figure 7 compares the commanded and measured linear velocity v_x for both firmware versions. The results show that linear velocity tracking remains largely unchanged after introducing gyrodometry. This behavior is expected, as the modification primarily affects yaw rate estimation and does not alter wheel speed control directly.

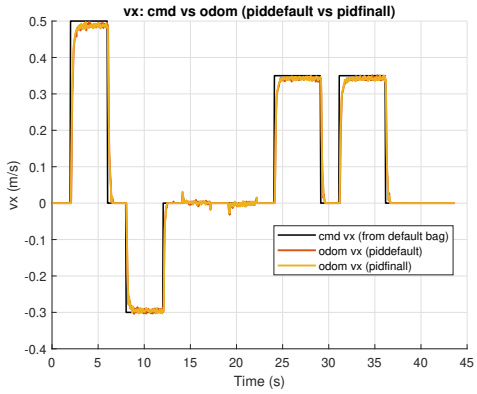

Fig. 7: Linear velocity tracking comparison between wheel-based odometry and gyrodometry-enhanced firmware. Both versions exhibit similar v_x tracking performance, indicating that the gyroscope integration does not affect translational control.

In contrast, Figure 8 highlights a clearer difference in angular velocity tracking. During in-place rotations and curved motion segments, the gyrodometry-enhanced firmware exhibits reduced angular velocity tracking error. Quantitative analysis shows a modest but consistent reduction in mean squared error (MSE) for w_z when using the gyroscope-based yaw rate.

D. Discussion

While the gyrodometry-enhanced firmware demonstrates improved angular velocity estimation, the overall performance difference remains moderate. This is largely attributed to the fact that the underlying PID controller was already well tuned, resulting in stable and consistent wheel speed regulation. As a result, the benefit of gyroscope integration appears primarily during high angular velocity maneuvers rather than steady-state motion.
Nevertheless, the observed reduction in yaw rate error improves rotational consistency and repeatability, which is beneficial for downstream navigation and localization

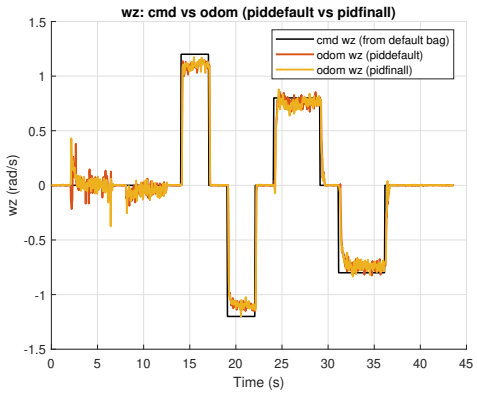

Fig. 8: Angular velocity tracking comparison between wheel-based odometry and gyrodometry-enhanced firmware. The gyroscope-based approach yields lower tracking error during rotational maneuvers, particularly for in-place turns and combined v_x–w_z motion.

tasks. The firmware modification provides a low-cost and modular improvement to odometry accuracy without modifying the control architecture, making it a suitable enhancement for more demanding motion scenarios.

IV. Motion Controller